

Technical Reference: Production-Grade AI Knowledge Base & Evaluation Platform

1. Executive Architecture Overarching Philosophy

In the current landscape of AI development, there is a significant industry gap between "toy" prototypes and production-ready systems. Most "naive RAG" (Retrieval-Augmented Generation) implementations fail in production because they lack a feedback loop; they are built on the hope that the LLM will find the right information, without any empirical method to prove it. This platform represents a fundamental shift from a "Document-to-Chat" interface to a **Document-to-Evaluation** architecture. By treating AI development as an engineering discipline rather than a prompting exercise, we replace subjective "vibes" with objective metrics. This approach mimics the rigorous requirements of professional AI teams who prioritize reliability, observability, and systematic optimization over simple fluency.

Comparison: Portfolio Projects vs. Production AI Development

Standard Portfolio Projects, Production AI Development Platform

Workflow: Upload -> Chat -> Answer, Workflow: Upload -> Retrieval -> Answer ->

Evaluation -> Metrics -> Experimentation

"Focus: Basic functionality and ""chat-like"" feel.", "Focus: Reliability, observability, and systematic optimization."

Philosophy: Success is measured by a single coherent response., Philosophy: Success is measured by statistical accuracy across thousands of queries.

Outcome: A standalone tool for casual use., "Outcome: A robust, observable system designed for iterative performance gains."

2. The Technical Stack: Rationale and Integration

The stack is curated to demonstrate mastery of the 2026 industry standards, emphasizing scalability and specialized RAG orchestration.

- **Frontend & UI (Next.js, Shadcn UI, Tailwind CSS):** Next.js is the non-negotiable industry standard for enterprise AI apps due to its server-side capabilities and deployment efficiency. We utilize Shadcn UI and Tailwind CSS to ensure a professional, modern aesthetic that recruiters recognize as the "enterprise look."
- **RAG Orchestration (LlamaIndex):** While LangChain is a generalist framework, **LlamaIndex** was selected for its specific focus on data indexing and retrieval. It provides superior data connectors and specialized index structures—such as **VectorStoreIndex**—which are essential for creating the structured "Document-to-Evaluation" flow this project demands.
- **LLM & Embeddings (OpenAI):** We leverage **GPT-5 or GPT-4.1** for high-level reasoning and complex query synthesis. For the vector layer, we utilize the **text-embedding-3-small** model, providing a cost-effective yet highly reliable solution for generating high-dimensional representations of text.
- **Data Layer (Supabase):** Supabase serves as our unified backend, providing a managed **PostgreSQL** instance with native **pgvector** support. This allows us to store relational metadata and vector embeddings in a single, high-performance environment.

3. The RAG Pipeline: From Ingestion to Generation

A production pipeline is more than a simple script; it is a multi-stage lifecycle designed to minimize noise and maximize retrieval accuracy.

1. **PDF Upload:** Documents are ingested via a secure interface and stored in Supabase Storage.
2. **Document Parsing:** Raw text is extracted while preserving essential metadata.
3. **Chunking:** Text is partitioned into manageable segments. This is a critical hyperparameter that determines the granularity of the AI's "memory."
4. **Embedding Generation:** Each chunk is converted into a vector using OpenAI's embedding models.
5. **Vector Database Storage:** Vectors are indexed within Supabase, enabling high-speed semantic search.

The Retrieval Process

When a user submits a query, the system performs a **Vector Search** to identify the **Nearest Neighbors**—the text chunks most mathematically relevant to the question. This retrieved context acts as the **Ground Truth** for the LLM, ensuring that the final answer is grounded in the provided data rather than the model's training weights.

4. The Evaluation Framework: Ragas Metrics

The **Evaluation Engine** is the core differentiator of this architecture. Instead of asking "Does this look right?", we use the Ragas framework to provide quantitative scores for every interaction. This shift is critical for measuring accuracy and reliability rather than mere linguistic fluency.

- **Faithfulness:** Measures the degree to which the answer is derived solely from the retrieved context. This is the primary defense against **Hallucinations**.
- **Answer Relevance:** Quantifies how well the response actually addresses the user's prompt.
- **Context Precision:** Evaluates the signal-to-noise ratio of the retrieval process, ensuring the most relevant chunks are ranked highest.
- **Context Recall:** Checks if the system successfully retrieved all necessary information required to answer the question, based on the **Ground Truth**.

5. Observability with Langfuse

In a production environment, an AI system cannot be a "black box." Recruiters value **Observability** as a marker of senior-level engineering. By integrating **Langfuse**, we gain total transparency into the LLM lifecycle. For every interaction, we record:

- The raw **User Question**.
- The specific **Retrieved Chunks** and their associated **Similarity Scores**.
- The exact **Prompt Template** and version used.
- The model's **Answer** and the corresponding **Evaluation Scores**. This tracing capability allows for "under-the-hood" troubleshooting and ensures that performance regressions are identified immediately.

6. The Experimentation Engine

The "Experimentation Engine" treats AI development as a science. To find the optimal configuration, the platform supports A/B testing across various system parameters.

- **Chunk Sizes:** Testing 200, 500, and 1000 token variations to balance context and precision.
- **Embedding Models:** Evaluating different models for vector representation.
- **Retrieval Methods:** Comparing standard vector search against hybrid search techniques.
- **Prompt Variations:** Iterating on system instructions to minimize bias and improve formatting. These results are aggregated into a **Leaderboard**, providing a data-driven path to the highest-performing system configuration.

7. Supabase Security and Infrastructure

A Senior Architect prioritizes the safety of user data. We utilize Supabase not just for storage, but as a comprehensive security layer.

- **Supabase Auth:** Provides production-ready user management and session handling.
- **Row Level Security (RLS):** This is the core security mechanism that ensures users can only access and query their own documents. We implement RLS policies at the PostgreSQL level to protect data at the source.
- **Unified Metadata:** We use Supabase Storage for PDFs and PostgreSQL for the associated metadata, ensuring a relational link between the raw file, the user, and the generated embeddings.

8. Development Roadmap: Phase-by-Phase Deliverables

Phase Name, Primary Goal, Key Technical Deliverable

1. Project Setup, Establish core infrastructure., Next.js/Supabase skeleton with deployed login system.
2. File Uploads, Implement document ingestion., Secure PDF upload to Supabase Storage with metadata.
3. Processing, Text extraction and chunking., "Database populated with clean, searchable text segments."
4. Embeddings, Vectorize knowledge., Functional Vector Search integration with OpenAI.
5. Retrieval, Context isolation., Query engine returning the most relevant chunks.
6. AI Answers, Grounded generation., RAG system generating answers based on retrieved context.
7. Evaluation, Objective quality control., "Automatic generation of Ragas scores (Faithfulness, etc.)."
8. Experimentation, Systematic optimization., Leaderboard comparing chunk sizes and prompt versions.
9. Observability, Full-stack tracing., Langfuse dashboard recording all traces and scores.
10. Portfolio Polish, Presentation of results., Professional case study with performance charts and diagrams.

9. Conclusion: Production-Quality Standards

The final output of this project is a comprehensive **Case Study** designed to meet 2026 hiring standards. By moving beyond the "chat" and into the "evaluation" space, this platform demonstrates the two most critical skills in modern AI engineering: **Observability** and **Empirical Evaluation**. While others build chatbots, this architecture builds a scientific framework for reliable intelligence. The resulting portfolio features—including the experimentation leaderboard, Ragas metrics, and Langfuse traces—provide definitive proof of a developer's ability to build, maintain, and optimize AI systems in a professional, enterprise-grade environment.